EXPERIMENT NO: 11

Experiment to understand pandas library use case in Data Science

Program:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
print(df)
df.columns=['Empd','Name','Salary']
print(df)
print(df.info())
df=pd.read_csv("50_Startups.csv")
print(df.info())
print(df.head())
print(df.tail())
df=pd.read_csv("employee.csv")
print(df.head())
print(df.tail())
print(df.info())
print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
print(df.salary.std())
print(df.describe())
print(df.describe(include='all'))
empCol=df.columns
print(empCol)
emparray=df.values
print(emparray)
employee_DF=pd.DataFrame(emparray,columns=empCol)
print(employee_DF)
```

Output:

```
  0      1      2
0  1  Smith  50000
1  2  Jones  60000
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
None
   R&D Spend  Administration  Marketing Spend       State     Profit
0   74308.32       180893.92         29300.28    New York   68422.31
1  157853.57       154642.93        304566.74    New York  166448.03
2  126139.12       176832.36        158031.97     Florida  123038.04
```

```
3  106805.48       170801.69        246399.66  California  123228.12
4   42622.70       130716.50        427942.75  California   88386.53
    R&D Spend  Administration  Marketing Spend       State     Profit
45  116065.73       116662.41         31783.56  California   94265.10
46   65198.11       120568.93        292351.77  California   93281.48
47   95409.86       107718.04        243718.96     Florida  111255.33
48   99272.99        53431.58         38422.83     Florida   80461.01
49   46803.90        64565.34        141784.14  California   58761.20
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp_id  5 non-null      int64
 1   name    5 non-null      object
 2   salary  5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 252.0+ bytes
None
0    5000
1    6000
2    7000
3    5000
```

```
4    8000
Name: salary, dtype: int64
<class 'pandas.core.series.Series'>
6200.0
6000.0
0    5000
Name: salary, dtype: int64
1700000.0
1303.8404810405298
         emp_id       salary
count  5.000000     5.000000
mean   3.000000  6200.000000
std    1.581139  1303.840481
min    1.000000  5000.000000
25%    2.000000  5000.000000
50%    3.000000  6000.000000
75%    4.000000  7000.000000
max    5.000000  8000.000000
         emp_id              name       salary
count   5.000000                 5     5.000000
unique       NaN                 5          NaN
top          NaN  SREE VARSINI K S          NaN
freq         NaN                 1          NaN
mean    3.000000               NaN  6200.000000
std     1.581139               NaN  1303.840481
min     1.000000               NaN  5000.000000
25%     2.000000               NaN  5000.000000
50%     3.000000               NaN  6000.000000
75%     4.000000               NaN  7000.000000
max     5.000000               NaN  8000.000000
Index(['emp_id', 'name', 'salary'], dtype='object')
[[1 'SREE VARSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
```

```
 [5 'SRI AKASH H G' 8000]]
```

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000